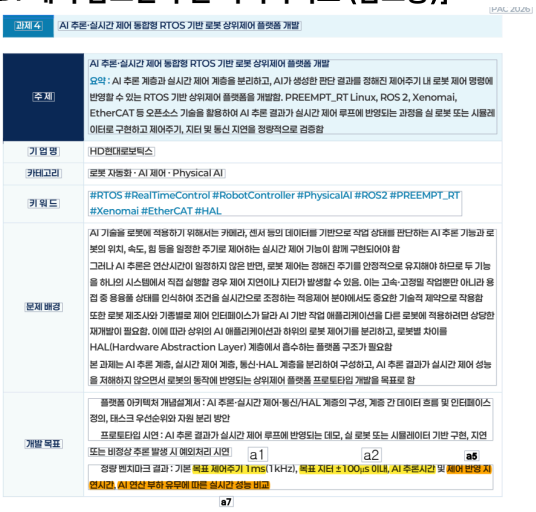
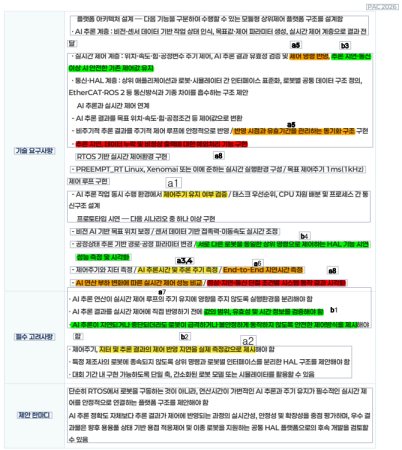
성능지표 정리

[PDF에서 참고한 부분 하이라이트 (참고용)]





[참고 논문, 자료 링크 정리]

▼ 참고자료 설명되어있는 표

먼저 한 가지 선을 긋겠습니다. **12개 지표 자체는 출제 PDF에서 나왔습니다.** A1~A8은 PDF가 이름을 직접 쓴 것이고, B1~B4는 PDF의 "필수 고려사항"을 숫자로 증명하기 위해 우리가 정의한 것입니다. 논문은 지표를 고르는 데 쓴 게 아니라, **각 지표를 "어떻게 정의하고, 재고, 보고할지"의 근거**로 썼습니다. 그 역할로 다시 정리하면 아래와 같습니다.

지표 정의·측정법의 근거가 된 논문

논문	어느 지표	가져온 것
Cerqueira & Brandenburg, "A Comparison of Scheduling Latency in Linux, PREEMPT_RT, and LITMUS^RT", OSPERT 2013	A2, A7	지연의 정의(깨어난 시각 − 깨어났어야 할 시각), cyclictest 동작, 부하 3종(유휴 / 캐시 압박 / 인터럽트 폭주) 설계, 히스토그램 + min/max/avg/median/stdev 보고 형식, cyclictest의 블랙박스 한계
Ye et al., "ROS2 Real-time Performance Optimization and Evaluation", Chinese J. Mech. Eng. 2023	A1, A2, A5, A6	1ms 주기의 두 가지 지터 정의(d_k 절대 지연, P_k 주기 흔들림), CLOCK_MONOTONIC + clock_nanosleep 관행, EtherCAT 1ms 주기 min/avg/max 보고 형식, 25.7시간 장시간 측정 근거, DDS vs 공유메모리 전달 지연
Hofer et al., "Industrial Control via Application Containers", arXiv 2005.01890, 2020	A2	"최대 지연은 주기의 1/10 이내"라는 100µs 기준의 근거, 최소 100만 회 반복, 초과 횟수·비율 보고
Teper et al., "End-To-End Timing Analysis in ROS2", RTSS 2022	A5, A6	reaction time과 data age의 공식 정의(우리 A6 = data age), 메시지 안에 origin 타임스탬프를 실어 체인 끝에서 빼는 측정법(우리 로그 포맷의 원형)
Günzel et al., "On the Equivalence of Maximum Reaction Time and Maximum Data Age", ECRTS 2023	A6	두 정의가 같으므로 data age 하나만 보고해도 된다는 근거 (결론만 인용)

논문	어느 지표	가져온 것
Yen et al., "Jiao: Bridging Isolation and Customization in Mixed Criticality Robotics", arXiv 2605.03641, 2026	A7, A1	부하 비교 표의 열(σ, p99, p99.9, max, excursion 횟수·비율, missed cycle), CCDF와 초당 excursion 그림 형식, "PREEMPT_RT만으로는 캐시·메모리 대역폭 경합을 못 막는다"는 CCX 격리 근거
Black, Galliker, Levine, "Real-Time Execution of Action Chunking Flow Policies", NeurIPS 2025	A3, A4, B2	추론 지연을 제어 주기 단위로 환산하는 정의(d = ⌊δ/Δt⌋), 과거 지연의 max로 유효기간을 잡는 근거, 지연을 늘려 가며 성능 저하를 재는 실험 설계
Ruan et al., "A Latency-Aware Framework for Visuomotor Policy Learning on Industrial Robots", arXiv 2602.14255, 2026	A3, B1, B2	지연을 관측/추론/실행 셋으로 분해하고 QR코드로 카메라 지연을 재는 방법, 만료 액션 폐기와 직전 자세 유지(hold) 관행, B2 지표 정의(RMS 저크, 힘 변화율, 접촉력), 지연 3단계 × 전략 3종 비교표

도구 문서 (논문은 아니지만 지표 형식의 근거)

자료	어느 지표	가져온 것
ros2-resilience (GitHub, ROS 2 Jazzy)	A8, B3	장애 시나리오 4종(dropout / latency / stale-hold / healthy), B3의 세 숫자 형식(감지 시간, 복구 명령 시간, 안전 상태 도달 시간), "자기 보고가 아니라 독립 스트림으로 복구 확인" 원칙
faultjector (GitHub)	A8	장애 종류 분류(delay / drop / duplicate / reorder / clock_skew), YAML 시나리오 형식, 허용 한계를 자동으로 찾는 sweep 개념. 단 아직 사용 불가 상태라 설계 참고만
MCIB 벤치마크 사양 (GitHub)	A7	AI 추론 vs 실시간 제어 간섭을 재는 방법론, "p99.9는 안전한 WCET 상한이 아니다". 데이터셋 미공개라 방법론 참고만

초록·요약만 참고한 것

- **Reghenzani, Massari, Fornaciari**, "The Real-Time Linux Kernel: A Survey on PREEMPT_RT", ACM Computing Surveys 2019. 1단계 학습 교과서로 추천했지만 사이트 봇 검증에 막혀 본문 미확인. 인용은 직접 확인 후에.
- **Puck & Keller**, "Performance Evaluation of Real-Time ROS2 Robotic Control in a Time-Synchronized Distributed Network", IEEE CASE 2021. 비실시간 인식 → 실시간 제어 구조에서 E2E를 잰 논문이라 A6 측정 지점 설계의 본보기이지만 IEEE 유료.

지표별로 뒤집어 보면

지표	1순위 근거	2순위 근거
A1	Ye 2023	Jiao
A2	Cerqueira & Brandenburg	Ye 2023, Hofer
A3	RTC	Ruan
A4	RTC	—
A5	Teper	Ye 2023
A6	Teper	Günzel
A7	Jiao	Cerqueira & Brandenburg
A8	ros2-resilience	faultjector
B1	Ruan	ros2-resilience
B2	Ruan	RTC
B3	ros2-resilience	Jiao
B4	전용 근거 없음 (후보: ros2 fanuc interface, Ye 2023 §4.8)	—

B4는 이번에 본문을 연 논문이 없어 근거가 가장 약합니다. 리포트에 인용이 필요하면 ros2 fanuc interface(arXiv 2506.14487, 오픈)를 다음에 읽는 게 좋습니다. 전체 상세는 validation/PAPERS_BY_METRIC.md에 있습니다.

https://www.semanticscholar.org/paper/A-Comparison-of-Scheduling-Latency-in-Linux%2C-and-RT-Cerqueira-Brandenburg/fb18f578592026e7b7d86b968971028830f2432e

https://link.springer.com/article/10.1186/s10033-023-00976-5

https://link.springer.com/article/10.1186/s10033-023-00976-5

Industrial Control via Application Containers:Maintaining...

Industry 4.0 is changing fundamentally data collection, its storage and analysis in industrial processes, enabling novel application such as flexible manufacturing of highly customized products....

https://arxiv.org/abs/2005.01890

https://daes.cs.tu-dortmund.de/storages/daes-cs/r/publications/teper_rtss_2022.pdf

On the Equivalence of Maximum Reaction Time and Maximum Data Age for Cause-Effect Chains

Real-time systems require a formal guarantee of timing-constraints, not only for individual tasks but also for data-propagation. The timing behavior of data-propagation paths in a given system is typically described by its maximum reaction time and its maximum data age. This paper shows that they are equivalent. To reach this conclusion, partitioned job chains are introduced, which consist of one immediate forward and one immediate backward job chain. Such

https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECRTS.2023.10

https://arxiv.org/abs/2605.03641

https://arxiv.org/abs/2506.07339

https://arxiv.org/abs/2602.14255

---

https://github.com/HarunTeper/ros2-end-to-end

https://drops.dagstuhl.de/storage/00lipics/lipics-vol262-ecrts2023/LIPIcs.ECRTS.2023.10/LIPIcs.ECRTS.2023.10.pdf

---

[PDF에서 참고한 부분 하이라이트 (참고용)]

[참고 논문, 자료 링크 정리]

1. 지켜야 할 제약 (측정이 측정을 망치지 않도록)

2. 제어루프 · AI 계층 로그 포맷 약속 (플랫폼 담당자와의 인터페이스)

- **1. 제어루프 로그 `_ctrl.csv` (매 주기 1행, 1kHz면 초당 1000행)**
- **2. AI 계층 로그 `_ai.csv` (추론 1회당 1행)**
- **3. 장애 주입 로그 `_fault.csv` (이벤트 1건당 1행)**
- **4. 파일 헤더**

3. 직관적인 설명

- 과제 4의 최종 목적 한 문장
- 약속 1: 실시간성을 증명하는 지표 (A1~A7)
- 약속 2와 3: 안정성과 확장성을 증명하는 지표 (A8, B1~B4)

4. 지표 별 구현법

- 먼저 전체 그림: 측정은 "일기장 3권 → 빼기 → 그림"
- 실시간성 지표 A1~A7: 예시 숫자로 직접 해 보기
- 안정성·확장성 지표 A8, B1~B4

a7 더 자세히 설명

- 왜 이런 실험을 하는가
- 네 가지 상황이 우리 컴퓨터에서 정확히 무엇인가
- 실제로 어떻게 돌리나: 터미널 두 개
- 결과를 어떻게 모으고 무엇이 "성공"인가

a8 해석법

그림 읽는 법: 위에서 아래로 네 칸

이 그림에서 숫자를 뽑는 법 (B2, B3)

이 예시가 보여 주는 교훈 하나

다섯 조건에서 각각 어떻게 다르게 보이나

1. 지켜야 할 제약 (측정이 측정을 망치지 않도록)

- 모든 시각은 `clock_gettime(CLOCK_MONOTONIC)`, 단위 **ns, 정수**. AI 프로세스와 제어 프로세스가 같은 시계를 써야 뺄셈이 성립한다.
- 제어루프 안에서 **파일 I/O 금지, 동적 할당 금지, printf 금지.** 메모리 **링버퍼**에 숫자만 쌓고, 종료 시 또는 별도 저우선순위 스레드가 CSV로 내보낸다. (Cerqueira & Brandenburg 2013 §2.4, Ye 2023 §3 관행)
- 제어루프 스레드: `SCHED_FIFO` 우선순위 99, `mlockall`, CPU 4 고정, `clock_nanosleep(TIMER_ABSTIME)`로 절대시각 대기. 상대 대기(`nanosleep(1ms)`)는 오차가 누적되므로 쓰지 않는다.
- 로그 CSV 첫 줄들은 `#` 메타데이터: `version, kernel, cmdline, governor, load, cmd, start`.

---

2. 제어루프 · AI 계층 로그 포맷 약속 (플랫폼 담당자와의 인터페이스)

성능측정이 가능하려면 플랫폼 코드가 아래 타임스탬프를 남겨야 한다. 모든 시각은 **같은 시계** `CLOCK_MONOTONIC`, 단위 **ns**, 정수. 루프 안에서는 파일에 쓰지 말고 메모리 링버퍼에 쌓았다가 종료 시(또는 저우선순위 스레드가) CSV로 내보낸다.

1. 제어루프 로그 `_ctrl.csv` (매 주기 1행, 1kHz면 초당 1000행)

열	타입	뜻
`iter`	int	반복 번호 (0부터)
`t_expected_ns`	int	이번 주기에 깨어났어야 할 절대 시각
`t_wake_ns`	int	실제 깨어난 시각 → **지연 = t_wake − t_expected**
`t_cmd_ns`	int	제어 명령을 HAL/시뮬레이터로 보낸 시각 → 루프 실행시간 = t_cmd − t_wake
`ai_seq`	int	이번 주기에 사용 중인 AI 결과의 일련번호 (없으면 -1)
`ai_t_capture_ns`	int	그 AI 결과의 센서 캡처 시각 (없으면 0)
`ai_t_recv_ns`	int	그 AI 결과를 제어루프가 처음 받은 시각 (없으면 0)
`ai_state`	str	`fresh` / `hold` / `decel` / `stop` / `reject` 중 하나 (예외처리 상태)
`q_ref`	float	기준 위치 (단일 축이면 1열, 다축이면 `q_ref_0..n`)
`q_act`	float	실측 위치
`v_cmd`	float	명령 속도

E2E 지연 = `t_cmd_ns(처음 반영된 주기) − ai_t_capture_ns` 반영 지연 = `t_cmd_ns(처음 반영된 주기) − ai_t_recv_ns`

2. AI 계층 로그 `_ai.csv` (추론 1회당 1행)

열	타입	뜻
`seq`	int	결과 일련번호 (제어루프의 `ai_seq` 와 매칭)
`t_capture_ns`	int	센서/이미지 캡처 시각
`t_infer_start_ns`	int	추론 시작
`t_infer_end_ns`	int	추론 끝 → 추론시간 = end − start
`t_publish_ns`	int	공유메모리/ROS 2 로 내보낸 시각
`valid_until_ns`	int	유효기간 만료 시각
`confidence`	float	신뢰도

추론 주기 = 연속된 `t_publish_ns` 의 차이

3. 장애 주입 로그 `_fault.csv` (이벤트 1건당 1행)

열	뜻
`t_ns`	주입 시각
`kind`	`delay` / `drop` / `outlier` / `disconnect` / `recover`
`param`	지연이면 ms, 이상값이면 값 등

4. 파일 헤더

CSV 첫 줄들은 `#` 로 시작하는 메타데이터:

```
# version=<플랫폼 git hash 또는 버전>
# kernel=<uname -r>
# cmdline=<cat /proc/cmdline>
# governor=performance
# load=idle|stress|ai-gpu|full
# cmd=<실행 명령>
# start=<ISO 날짜시각>
```

[개발자] 실행 가능한 빌드 + 로그 3종 + 버전 식별자 전달
▼
[성능] configs/ 조건 기록 → 체크리스트 → 측정 → raw/ 저장 → scripts/ 분석 → figures/ → reports/
▼
[팀] 리포트 리뷰 → 설계 수정 → 다음 사이클

---

3. 직관적인 설명

과제 4의 최종 목적 한 문장

"생각이 느리고 들쭉날쭉한 AI(머리)와, 1초에 1000번 박자를 절대 놓치면 안 되는 로봇 제어(손)를 한 시스템 안에서 안전하게 연결하는 구조를 만들고, 그것이 정말 작동한다는 걸 숫자로 증명하라."

여기서 심사위원이 확인하고 싶은 약속은 세 가지입니다. 과제 PDF의 "제안 한마디"에 나오는 단어 그대로입니다.

- **실시간성**: 머리가 아무리 바빠도 손의 박자는 흐트러지지 않는가?
- **안정성**: 머리가 늦거나, 틀리거나, 말을 끊어도 손이 위험한 짓을 안 하는가?
- **확장성**: 다른 로봇으로 바꿔도 머리 쪽 코드는 그대로인가?

12개 지표는 이 세 약속을 각각 **"말"이 아니라 "숫자"로 바꾸는 장치**입니다. 지표가 없으면 "우리 시스템은 안정적입니다"라는 주장만 남고, 심사위원은 믿을 근거가 없습니다.

약속 1: 실시간성을 증명하는 지표 (A1~A7)

A1 제어주기 유지(데드라인 미스)와 A2 지터는 "손이 박자를 지키는가"입니다.

이 과제의 전제 자체가 "로봇 제어는 1ms마다 정확히 명령을 내야 한다"입니다. 그래서 가장 먼저 증명해야 하는 건 우리 손이 정말 1초에 1000번, 매번 100µs 오차 안에 깨어나는가입니다. A1은 "박자를 아예 빼먹은 적이 있나"(있으면 안 됨), A2는 "박자마다 얼마나 늦었나"(최대 100µs)입니다. 왜 둘로 나누냐면, 조금씩 늦는 것과 한 박자를 통째로 놓치는 것은 로봇에 미치는 결과가 다르기 때문입니다. 조금 늦으면 정밀도가 떨어지고, 통째로 놓치면 로봇이 덜컹합니다. 이 둘이 없으면 나머지 지표는 의미가 없습니다. 박자를 못 지키는 손에 머리를 연결해 봐야 소용없으니까요.

A3 추론시간과 A4 추론 주기는 "머리가 얼마나 느리고 얼마나 자주 말하는가"입니다.

이 과제의 문제 배경이 "AI 추론은 연산시간이 일정하지 않다"입니다. 그 "일정하지 않음"을 숫자로 보여 줘야 우리가 푼 문제의 크기가 드러납니다. 머리가 평균 10ms, 최악 50ms 걸린다면 손은 그 사이 10~50번의 박자를 머리 없이 쳐야 합니다. A3는 그 "머리 없는 시간"의 길이이

고, A4는 그런 공백이 얼마나 자주 오는지입니다. 이 두 숫자가 있어야 "유효기간을 몇 ms로 둘지", "예상 지연만큼 미래를 겨냥한다"는 팀 설계의 근거가 생깁니다. 과제 기준값은 없지만, 이게 없으면 뒤의 A5·A6가 왜 그 값인지 설명이 안 됩니다.

A5 제어 반영 지연은 "머리의 말이 손에 전달되는 데 걸리는 시간"입니다.

과제 문서가 이 지표를 세 번이나 언급한 이유는, 이것이 **우리가 만든 "연결 구조"의 직접 성적표**이기 때문입니다. A1·A2는 손 자체의 성능이고 A3·A4는 머리 자체의 성능입니다. 둘 다 좋아도 연결이 나쁘면 과제는 실패입니다. 머리의 결과가 제어 계층 문 앞에 도착했는데 손이 그걸 받아 쓰기까지 수십 ms가 걸리면, "비주기적 추론 결과를 주기적 제어 루프에 안정적으로 반영하는 동기화 구조"라는 요구를 못 채운 겁니다. 이 지표가 작고 흔들림이 없다는 건 공유메모리, 검증, 반영 정책이라는 우리 설계가 제대로 돌아간다는 유일한 증거입니다.

A6 End-to-End 지연은 "로봇이 얼마나 과거의 세상을 보고 움직이는가"입니다.

카메라가 찍은 순간부터 로봇 명령이 바뀌는 순간까지입니다. 용접으로 치면 "지금 조정하는 조건이 몇 ms 전 용융풀 상태 기준인가"입니다. 이 숫자가 커지면 아무리 박자를 잘 지켜도 로봇은 이미 지나간 상황에 반응합니다. A6는 A3(추론)과 A5(반영)를 포함한 전체 합이라, 어느 구간이 병목인지 쪼개서 보여 주면 "다음에 뭘 개선해야 하는지"까지 말할 수 있습니다. 과제가 요구하는 “AI 추론시간 및 제어 반영 지연시간”의 종합판입니다.

A7 AI 부하 유무 비교는 "머리가 바쁠 때 손이 흔들리는가"입니다.

이것이 사실상 **과제 4의 핵심 질문**입니다. 문제 배경에 "두 기능을 하나의 시스템에서 실행할 경우 제어 지연이나 지터가 발생할 수 있음"이라고 적혀 있고, 필수 고려사항 첫 줄이 “AI 추론 연산이 제어 루프의 주기 유지에 영향을 주지 않도록 실행환경을 분리해야 함”입니다. 그러니 A1·A2를 “AI가 없을 때”와 “AI가 세게 돌 때” 양쪽에서 재서 나란히 놓아야 합니다. 둘이 같으면 분리에 성공한 것이고, 부하 쪽이 나빠지면 분리가 뚫린 것입니다. 팀의 베이스라인이 유휴 146µs에서 부하 1325µs로 뛴 것이 바로 "분리가 안 된 상태"의 모습이고, 우리가 보여 줄 before/after의 before입니다.

약속 2와 3: 안정성과 확장성을 증명하는 지표 (A8, B1~B4)

A8 조건별 동작 시각화는 "머리가 고장 났을 때 손이 어떻게 하는지 보여 주는 영상"입니다.

과제는 "지연 또는 비정상 추론 발생 시 예외처리 시연"을 산출물로 요구하고, 필수 고려사항에 “AI 추론이 지연되거나 중단되더라도 로봇이 급격하거나 불안정하게 동작하지 않도록”이라고 못 박았습니다. 정상, 지연, 누락, 이상값, 통신단절 다섯 상황에서 로봇의 위치와 속도가 어떻게 변했는지 시간축 그래프로 보여 주는 게 A8입니다. 이건 "시연"에 가장 가까운 지표라 심사위원이 가장 직관적으로 이해합니다. 그림 한 장에서 속도가 부드럽게 줄고 다시 부드럽게 이어지면 설명이 필요 없습니다.

B1 검증 거부율은 "손이 머리 말을 무조건 믿지 않는다는 증거"입니다.

필수 고려사항에 “AI 추론 결과를 반영하기 전에 값의 범위, 유효성 및 시간 정보를 검증해야 함”이 있습니다. 그런데 검증 코드가 있다는 것만으론 증명이 안 됩니다. 일부러 범위 밖 값, 오래된 값, 신뢰도 낮은 값을 넣어서 "몇 개 중 몇 개를 걸렀는가"를 세야 검증이 **실제로 작동한다**는 증거가 됩니다. 특히 "유효기간 지난 결과를 쓴 횟수 0"은 시간 검증이 뚫리지 않았다는 뜻이라 꼭 필요합니다. 과제 문서가 지표 이름을 주지 않아서 우리가 만든 것이지만, 이 요구사항을 숫자로 증명할 다른 방법이 없습니다.

B2 안전 동작(저크)은 "급격하지 않다"를 숫자로 바꾼 것입니다.

과제는 "급격하거나 불안정하게 동작하지 않도록"이라고만 했지 기준을 안 줬습니다. A8 그림만 내면 "부드러워 보인다"는 인상에 그칩니다. 그래서 위치를 미분해 저크(얼마나 갑자기 속도가 바뀌는가)를 구하고, 장애 전·중·후의 최대 저크를 비교합니다. 장애 중 저크가 정상 때와 비슷하면 "급격하지 않다"가 수치로 성립합니다. 자동차 승차감을 "편안해 보인다"가 아니라 "최대 저크 몇 m/s^3"으로 말하는 것과 같습니다.

B3 장애 복구 시간은 "고장을 얼마나 빨리 알아채고 얼마나 빨리 돌아오는가"입니다.

기술 요구사항에 "추론 지연·통신 이상 시 안전한 기존 제어값 유지"가 있습니다. 이걸 증명하려면 장애가 시작된 순간부터 감지, 감속, 정지, 복귀까지 각각 몇 ms 걸렸는지와, **그 와중에도 박자(A1)를 놓치지 않았는지**를 보여 줘야 합니다. 예외처리 코드가 무거워서 복구 중에 주기를 놓치면 안전 장치가 오히려 위험을 만든 셈이라, A1과 묶어서 보는 게 중요합니다.

B4 HAL 교체 비용은 "로봇을 바꿔도 머리는 그대로"의 증거입니다.

문제 배경의 두 번째 문제가 "로봇 제조사와 기종별로 제어 인터페이스가 달라 재개발이 필요함"이고, 그 해법이 HAL(통역사 계층)입니다. "서로 다른 로봇을 동일한 상위 명령으로 제어하는 HAL 기능 시연"이 요구되는데, 시연만으론 "진짜 분리됐는지" 알 수 없습니다. 로봇을 바꿨을 때 HAL 밖에서 바뀐 코드가 0줄이고, HAL이 명령을 변환하는 데 몇 µs밖에 안 걸려 A2를 해치지 않는다는 두 숫자가 있어야 확장성 약속이 증명됩니다.

마지막으로 전체를 한 줄씩 묶으면 이렇습니다. A1·A2는 손이 멀쩡한가, A3·A4는 머리가 얼마나 느린가, A5·A6는 둘의 연결이 빠른가, A7은 머리가 바빠도 손이 안 흔들리는가, A8·B1·B2·B3는 머리가 고장 나도 손이 안전한가, B4는 손을 바꿔도 머리가 그대로인가. 12개가 전부 있어야 "실시간성·안정성·확장성" 세 약속에 빈칸이 안 생깁니다.

4. 지표 별 구현법

먼저 전체 그림: 측정은 "일기장 3권 → 빼기 → 그림"

플랫폼이 돌아가는 동안 컴퓨터는 세 권의 일기장을 씁니다. 전부 그냥 표(CSV)이고, 엑셀로 열면 아래처럼 생겼습니다. 실제 파일은 시각이 나노초(ns)인데, 여기서는 읽기 쉽게 **마이크로초(µs)** 로 적겠습니다.

일기장 1. 제어루프 일기 (`ctrl.csv`): 손이 박자 한 번 칠 때마다 한 줄.

iter	t_expected	t_wake	t_cmd	ai_seq	ai_t_capture	ai_t_recv	ai_state
0	0	3	120	7	−20000	−300	fresh
1	1000	1004	1125	7	−20000	−300	fresh
2	2000	2002	2118	**8**	−5000	1600	fresh
3	3000	**3150**	3270	8	−5000	1600	fresh
4	4000	**5100**	5220	8	−5000	1600	fresh

열의 뜻은 이렇습니다. `t_expected`는 "깨어났어야 할 시각", `t_wake`는 "실제 깨어난 시각", `t_cmd`는 "로봇에 명령 보낸 시각". `ai_seq`는 "지금 쓰고 있는 머리의 말이 몇 번째 말인가". `ai_t_capture`는 그 말의 바탕이 된 카메라 사진이 찍힌 시각, `ai_t_recv`는 그 말이 손에게 도착한 시각. `ai_state`는 손의 상태(fresh=새 말 쓰는 중, hold=옛 말 유지, decel=감속, stop=정지, reject=말 무시). `q_ref`·`q_act`·`v_cmd`는 목표 위치, 실제 위치, 명령 속도.

일기장 2. 머리 일기 (`ai.csv`): 머리가 한 번 생각할 때마다 한 줄.

seq	t_capture	t_infer_start	t_infer_end	t_publish	valid_until	confidence
7	−20000	−19000	−9500	−9400	40600	0.90
8	−5000	−4000	1400	1550	51550	0.80

사진 찍힌 시각, 생각 시작, 생각 끝, 말한 시각, 이 말의 유효기간, 자신감.

일기장 3. 장애 일기 (`fault.csv`): 우리가 일부러 고장을 낼 때마다 한 줄.

t	kind	param
30000	drop	200ms
230000	recover	

"3만 µs 시점에 머리 말을 200ms 동안 끊었다, 23만 µs에 복구했다."

이 세 표만 있으면 12개 지표는 전부 **열끼리 빼고, 조건에 맞는 줄을 세는 것**으로 나옵니다. 그 빼기와 세기를 파이썬 스크립트가 하고, 사람은 결과 표와 그림을 봅니다. 파이썬에서 쓸 "계산기"는 하나만 만들어 두면 됩니다. 숫자 목록을 넣으면 최소, 평균, p50, p99, p99.9, 최대, 그리고 "기준 넘은 개수"를 돌려주는 함수입니다. 모든 지표가 이 계산기를 돌려 씁니다.

실시간성 지표 A1~A7: 예시 숫자로 직접 해 보기

A1 박자를 빼먹은 횟수

일기장 1에서 `t_wake − t_expected`를 줄마다 계산합니다. 위 표로 하면 3, 4, 2, 150, **1100**. 이 중 1000(=1ms)을 넘는 줄을 셉니다. 4번 줄 하나. 그러니 "5번 중 1번 미스"입니다. 실제로는 60만 줄에서 0번이어야 합니다. 결과물은 표 한 줄 "미스 횟수 1 / 전체 5 / 미스율 20%"와, 미스가 난 줄 번호 목록입니다.

A2 박자마다 얼마나 늦었나

A1에서 구한 똑같은 숫자들(3, 4, 2, 150, 1100)을 계산기에 넣습니다. 최대 1100, 100 넘는 개수 2. 이 숫자 목록으로 히스토그램을 그립니다. 가로축은 "몇 µs 늦었나", 세로축은 "그런 줄이 몇 개"인데 세로축을 로그로 둡니다. 3µs짜리가 59만 개고 1100µs짜리가 1개면 보통 눈금에선 1개짜리가 안 보이기 때문입니다. 100µs 자리에 빨간 세로선을 긋고, 그림 위에 "min 2 / max 1100 / avg 252 / p99 … / 샘플 5"를 한 줄 적습니다.

이 지표만 **두 번** 잽니다. 한 번은 우리 제어루프 일기로(위 방식), 한 번은 cyclictest라는 기성 프로그램으로. cyclictest는 "아무 일도 안 하고 박자만 치는 빈 손"이라서, 이 컴퓨터와 커널 설정이 낼 수 있는 가장 좋은 점수를 보여 줍니다. 실행은 한 줄입니다.

```
cyclictest -m -S -p 99 -i 1000 -h 400 -q -a 4 -t 1 -D 600 > raw/20261009_rt_ccx_idle_600s_r1.
txt
```

뜻은 "메모리 고정하고(-m), 우선순위 최고로(-p 99), 1ms마다(-i 1000), CPU 4번에서(-a 4) 스레드 하나로(-t 1), 10분 동안(-D 600), 결과는 히스토그램으로(-h 400)". 나온 파일은 행이 µs, 열이 CPU 번호, 값이 횟수인 표라서 5번째 열(CPU4)만 뽑으면 위와 똑같은 히스토그램이 됩니다. **빈 손 점수(cyclictest)와 실제 점수(제어루프)를 나란히 두면, 둘의 차이가 "우리 코드가 더한 지연"입니다.** 지금 폴더에 있는 `docs/benchmarks/baseline_generic_idle.txt`가 바로 이 cyclictest 출력이고, 이걸 그림으로 그리는 게 첫 실습입니다.

A3 머리가 한 번 생각하는 시간

일기장 2에서 `t_infer_end − t_infer_start`. 7번 말은 −9500 − (−19000) = 9500µs = 9.5ms, 8번 말은 1400 − (−4000) = 5400 = 5.4ms. 이 숫자들을 계산기에 넣어 평균, p99, 최대를 봅니다. 주의할 점 하나는 프로그램 켜고 처음 수십 번은 GPU가 "예열" 중이라 유독 느리니 버리고 셉니다. 결과물은 히스토그램과 표, 그리고 표 아래에 "유효기간 설정값 = 50ms, 측정 최대 = 9.5ms"처럼 설계값과 나란히 적습니다.

A4 머리가 얼마나 자주 말하나

일기장 2에서 이웃한 줄의 `t_publish`를 뺍니다. 1550 − (−9400) = 10950µs ≈ 11ms. "약 11ms마다 한 번 말한다"는 뜻이고, 손은 그 사이 11박자를 직전 말로 칩니다. 이 간격들을 계산기에 넣고, 평균의 3배 넘는 간격이 있으면 "말이 끊긴 구간"으로 시각을 기록해 둡니다.

A5 말이 도착해서 손이 쓰기까지

일기장 1에서 `ai_seq`가 **바뀐 줄**만 찾습니다. 위 표에선 2번 줄(7→8). 그 줄에서 `t_cmd − ai_t_recv` = 2118 − 1600 = **518µs**. "8번 말이 1600에 도착했고, 손이 2118에 처음 그 말대로 명령을 보냈다, 그 사이 0.5ms." 말이 바뀔 때마다 숫자가 하나씩 나오므로 이것들을 계산기에 넣습니다. 기대값은 "거의 항상 0.5~2ms". 가끔 수십 ms가 나오면 그 줄의 `t_cmd − t_wake`(손이 일한 시간)를 같이 봐서 검증·블렌딩 코드가 느렸던 건지 확인합니다.

A6 사진 찍힌 순간부터 로봇이 움직이기까지

A5와 같은 줄(2번)에서 이번엔 `t_cmd − ai_t_capture` = 2118 − (−5000) = **7118µs ≈ 7.1ms**. "로봇은 7.1ms 전 세상을 보고 움직였다." 여기에 일기장 2의 8번 줄을 붙여 네 구간으로 쪼갭니다.

구간	계산	값
사진→생각 시작	−4000 − (−5000)	1.0ms
생각 (A3)	1400 − (−4000)	5.4ms
말 전달	1600 − 1550	0.05ms
손이 쓰기까지 (A5)	2118 − 1600	0.52ms
합계 (A6)		7.1ms

이걸 누적 막대 하나로 그리면 "어디가 제일 큰가"가 한눈에 보입니다. 보통 "생각"이 가장 크고, "전달"이 수 ms면 통신 구조가 잘못된 겁니다.

A7 머리가 바쁠 때 손이 흔들리나

새로운 계산은 없습니다. **A1과 A2를 네 가지 상황에서 반복**해서 표 한 장에 모읍니다.

상황	어떻게 만드나
idle	아무것도 안 띄움
stress	컴퓨터 괴롭히기 프로그램. `stress-ng --vm 8 --vm-bytes 2G --timeout 600` 은 "8개 작업이 각각 2GB 메모리를 마구 읽고 쓰게" 해서 캐시를 못 쓰게 만듦
ai-gpu	실제 AI 모델을 GPU에서 계속 돌림
full	AI + 시뮬레이터 + ROS 2 전부

주의할 건 괴롭히기 프로그램을 **제어 코어가 아닌 코어**에서 띄우는 것(`taskset -c 8-15,24-31`을 앞에 붙임). 결과물은 상황별로 한 줄씩 있는 표(σ, p99, p99.9, max, 100µs 초과 횟수, 미스)와, 네 상황의 히스토그램을 한 그림에 겹친 것, 그리고 "초당 100µs 초과 횟수"를 시간순으로 그리고 부하를 켠 시점에 세로선을 그은 그림입니다. 부하를 켜도 선이 안 튀면 성공입니다. 각 상황을 최소 3번씩 반복하고 같은 시간(10분)으로 맞춥니다.

안정성·확장성 지표 A8, B1~B4

A8 고장 났을 때 손이 뭘 하는지 그림

먼저 "고장 내는 장치"가 필요합니다. 머리가 말을 손에게 넘기는 길목에 끼워 넣는 작은 프로그램으로, 설정에 따라 말을 늦게 넘기거나(delay), 한동안 안 넘기거나(drop), 엉뚱한 값으로 바꾸거나(outlier), 길 자체를 끊습니다(disconnect). 그리고 그럴 때마다 일기장 3에 한 줄 씁니다. 이 장치는 플랫폼 담당자와 "어디에 끼울지"를 정해야 하므로 선행 작업입니다.

그 다음 정상 포함 5가지 상황을 각각 돌려 일기장 1과 3을 받고, 상황당 그림 한 장을 그립니다. 위에는 머리 말이 도착한 시각을 점으로, 고장 구간을 회색 띠로. 가운데는 `ai_state`를 색 띠로(fresh 초록, hold 노랑, decel 주황, stop 빨강). 아래는 `q_ref`와 `q_act` 선, `v_cmd` 선. 고장 띠 안에서 속도 선이 완만하게 내려가고 복구 뒤 점프 없이 이어지면 됩니다.

B1 손이 말을 거른 비율

일기장 1에서 `ai_state == reject`인 줄을 셉니다. 분모는 일기장 2의 줄 수(도착한 말 총수). 예를 들어 100번 말했는데 3번 거부면 3%. 왜 거부됐는지(범위 밖 / 너무 오래됨 / 신뢰도 낮음)를 알아야 하니 플랫폼 담당자에게 사유 열 하나를 추가해 달라고 합니다. 그리고 특별 검사 하나를 더 합니다. 손이 말을 쓴 모든 줄에서 `t_cmd`가 그 말의 `valid_until`보다 큰지 확인합니다. 하나라도 있으면 "유효기간 지난 말을 썼다"는 뜻이라 0이어야 합니다. 이상값 주입 상황에서는 "우리가 넣은 엉뚱한 값 개수 = 거부된 개수"인지 맞춰 봅니다.

B2 덜컹거림 숫자

위치 `q_act` 열에서 이웃 줄끼리 빼면 속도, 그 속도를 또 이웃끼리 빼면 가속도, 한 번 더 빼면 저크입니다. 예를 들어 위치가 10.00, 10.04, 10.09, 10.14라면 속도는 0.04, 0.05, 0.05(1ms당), 가속도는 0.01, 0.00, 저크는 −0.01. 그런데 1ms 간격 숫자를 세 번 빼면 작은 측정 노이즈가 엄청 커져서 그래프가 지저분해집니다. 그래서 둘 중 하나를 합니다. 빼기 전에 "흔들림 평탄화"(저역통과 필터)를 걸거나, 실제 위치 대신 명령 속도 `v_cmd` 열을 빼서 가속도·저크를 구합니다. 어느 쪽을 했는지 리포트에 적습니다. 그 다음 일기장 3의 시각으로 "고장 전 / 중 / 후"를 나눠 각 구간의 최대 저크를 비교합니다. 고장 중 최대 저크가 평소와 비슷하면 "급격하지 않다"가 숫자로 성립합니다. 추종 오차는 `|q_ref − q_act|`의 최대값과 평균입니다.

B3 고장을 알아채고 돌아오는 데 걸린 시간

일기장 3에서 고장 시작 시각 t0(=30000)를 읽고, 일기장 1에서 `ai_state`가 바뀌는 줄들의 `t_wake`를 찾습니다. 예를 들어 fresh→hold가 80000, hold→decel이 85000, decel→stop이 150000, 새 말 도착(`ai_t_recv`)이 232000, stop→fresh가 240000이라면

구간	계산	값
감지까지	80000 − 30000	50ms
감속 시작~정지	150000 − 85000	65ms
새 말 도착~정상 복귀	240000 − 232000	8ms

그리고 30000~240000 사이에 A1 미스가 몇 번인지 셉니다(0이어야 함). "감지 50ms"는 유효기간 설정값과 비슷해야 정상입니다. 같은 실험을 5번 반복해 평균과 최대를 적습니다.

B4 로봇을 바꿨을 때 뭐가 바뀌었나

시뮬레이터에서 로봇 A용 설정과 로봇 B용 설정을 만들고, 둘 사이에 바뀐 코드를 `git diff --stat`으로 봅니다. 바뀐 파일이 전부 HAL 폴더 안이면 "HAL 밖 변경 0줄". 그리고 HAL이 상위 명령을 받아 로봇 명령으로 바꾸는 함수의 들어가는 시각과 나오는 시각을 찍어 달라고 해서 그 차이(보통 수 µs)를 계산기에 넣습니다. 마지막으로 같은 명령 시퀀스로 두 로봇을 돌려 A2가 비슷한지 봅니다.

누가 뭘 하나. 플랫폼 담당자가 하는 건 ① 일기장 세 권을 약속한 열대로 남기기 ② reject 사유 열과 HAL 타임스탬프 추가 ③ 고장 장치 끼울 자리 정하기. 당신이 하는 건 ① cyclictest·stress-ng 돌려서 A2·A7의 idle/stress 결과 먼저 만들기(모델 없이 지금 가능) ② "계산기" 함수와 그림 스크립트 만들기 ③ 일기장이 오면 열 이름 맞춰 돌리고 표·그림을 리포트에 넣기. 가장 먼저 할 일은 이미 있는 베이스라인 파일로 A2 히스토그램 한 장을 그려 보는 것입니다.

---

a7 더 자세히 설명

A7은 "새로운 계산"이 아니라 **"같은 측정을 네 번, 주변 상황만 바꿔서"** 하는 겁니다. 왜 그렇게 하는지부터, 우리 컴퓨터에서 정확히 어떻게 하는지까지 풀어 보겠습니다.

왜 이런 실험을 하는가

과제의 핵심 주장은 "AI가 아무리 바빠도 제어 박자는 안 흔들린다"입니다. 이걸 증명하는 방법은 하나뿐입니다. **AI가 쉴 때와 바쁠 때 박자를 각각 재서 나란히 놓는 것.** 두 숫자가 같으면 증명 끝, 바쁠 때 숫자가 나빠지면 "분리가 뚫렸다"는 뜻입니다.

그런데 "바쁘다"에도 종류가 있습니다. 컴퓨터 안에서 AI가 제어 루프를 방해하는 경로는 여러 갈래입니다.

- CPU 시간을 뺏는다 (같은 코어를 쓰려고 할 때)
- 캐시를 뺏는다 (같은 L3 캐시를 공유할 때, AI가 큰 데이터를 쓰면 제어 루프의 데이터가 밀려남)
- 메모리 통로를 막는다 (AI가 RAM을 마구 읽으면 제어 루프의 읽기가 줄 서야 함)
- 인터럽트를 쏟는다 (GPU 드라이버, 디스크, 네트워크가 "다 됐어요"라고 CPU를 자꾸 부름)

경로마다 막는 방법이 다릅니다. 코어 격리는 첫 번째를 막고, CCX 통째 격리는 두 번째를 막고, IRQ 친화도는 네 번째를 막습니다. 그래서 부하를 한 종류만 걸면 "다른 경로는 뚫려 있는데 모르고 지나갈" 수 있습니다. 네 가지 상황은 이 경로들을 하나씩 건드려 보는 겁니다.

네 가지 상황이 우리 컴퓨터에서 정확히 무엇인가

우리 컴퓨터의 코어 배치를 먼저 떠올리세요. ENVIRONMENT.md 기준입니다.

코어	용도
0~3, 16~19	운영체제, 인터럽트 처리
4, 20	**제어 루프 전용 (격리됨)**
5~7, 21~23	예비 격리 코어
8~15, 24~31	AI, 시뮬레이터, ROS 2, 그 외 전부

측정 대상은 항상 **CPU 4번에서 도는 제어 루프(또는 cyclictest)** 입니다. 바꾸는 건 "나머지 코어에서 무엇이 돌고 있나"뿐입니다.

상황 1: idle (유휴)

나머지 코어에 아무것도 안 띄웁니다. 터미널 하나만 열고 cyclictest를 돌립니다. 이게 "이 컴퓨터가 낼 수 있는 최고 점수"입니다. 다른 세 상황의 비교 기준점이라 꼭 필요합니다.

상황 2: stress (인위 부하)

`stress-ng`라는 "컴퓨터 괴롭히기 전용 프로그램"을 **8~15, 24~31번 코어에서** 띄웁니다. AI 모델이 아직 없어도 할 수 있고, 어떤 종류의 괴롭힘인지 내가 정할 수 있다는 게 장점입니다. 종류별로 옵션이 있습니다.

괴롭힘 종류	옵션	뚫으려는 경로
메모리를 마구 읽고 씀	`--vm 8 --vm-bytes 2G`	캐시, 메모리 통로
캐시를 일부러 어지럽힘	`--cache 4`	캐시
디스크를 쉴 새 없이 씀	`--hdd 2`	인터럽트
그냥 CPU만 100%	`--cpu 16`	CPU 시간 (격리됐으면 영향 거의 없어야 함)

논문(Cerqueira & Brandenburg)이 "그냥 CPU 100%"가 아니라 "L2보다 큰 메모리를 무작위로 읽는 부하"를 쓴 이유가 여기 있습니다. 단순 연산은 캐시 안에서 끝나서 아무도 방해하지 않습니다. 그래서 `--vm`에 큰 용량을 주는 겁니다.

상황 3: ai-gpu (실제 AI 추론)

팀의 실제 추론 모델을 GPU에서 쉬지 않고 돌립니다. stress-ng와 뭐가 다르냐면, GPU 추론은 stress-ng가 못 만드는 간섭을 만듭니다. GPU로 데이터를 보내고 받는 PCIe 통로 사용, GPU 드라이버가 "완료됐다"고 CPU를 부르는 인터럽트, CUDA 런타임 스레드의 메모리 접근. Jiao 논문이 "PREEMPT_RT만으로는 캐시와 메모리 대역폭 경합을 못 막는다"고 한 게 이런 종류입니다. 그래서 stress-ng로 통과해도 실제 AI로 한 번 더 확인해야 합니다. 모델이 나오기 전엔 못 하지만, 임시로 아무 PyTorch 모델(예: ResNet 추론 무한 반복)을 돌려도 같은 종류의 간섭은 만들어집니다.

상황 4: full (전부)

실제 데모와 같은 상태입니다. AI + MuJoCo 시뮬레이터 + ROS 2 노드 + 공유메모리 통신까지 전부 켜고 제어 루프를 돌립니다. 이게 심사위원 앞에서 보여 줄 조건이므로 최종 숫자는 여기서 나와야 합니다.

실제로 어떻게 돌리나: 터미널 두 개

모든 상황의 실행 방식은 똑같습니다. **터미널 A에서 부하를 띄우고, 터미널 B에서 측정을 돌린다.** 순서가 중요합니다. 부하를 먼저 켜서 안정시킨 뒤(30초 정도) 측정을 시작하고, 측정이 끝난 뒤 부하를 끕니다.

터미널 A (부하 담당). 핵심은 맨 앞의 `taskset -c 8-15,24-31`입니다. "이 프로그램은 이 코어들에서만 돌아라"는 뜻이고, 이게 없으면 부하가 CPU 4번까지 침범해서 실험이 무의미해집니다.

```
taskset -c 8-15,24-31 stress-ng --vm 8 --vm-bytes 2G --cache 4 --timeout 700
```

터미널 B (측정 담당). 30초 기다린 뒤 실행합니다. 파일명에 상황을 적어 둡니다.

```
cyclictest -m -S -p 99 -i 1000 -h 400 -q -a 4 -t 1 -D 600 > raw/20261009_rt_ccx_stress_600s_r1.txt
```

idle 상황이면 터미널 A를 안 쓰고 B만 돌리면 됩니다. ai-gpu 상황이면 터미널 A에서 stress-ng 대신 AI 추론 스크립트를 `taskset -c 8-15,24-31 python infer_loop.py`처럼 띄웁니다. full 상황이면 터미널 A 대신 실제 데모 실행 절차를 밟고, 측정은 cyclictest가 아니라 **제어루프 일기(ctrl.csv)** 에서 뽑습니다.

시간 흐름으로 보면 이렇습니다.

```
0초        30초                                  630초  700초
| 부하 켜기 |◄──────── 측정 10분 (cyclictest 또는 제어루프) ────────►| 부하 끔 |
            측정 파일 1개 = 상황 1개 × 회차 1개
```

확인할 것 두 가지. 부하가 정말 다른 코어에서만 도는지 `top`을 열어 `1`을 누르면 코어별 사용률이 나오는데, CPU 4번이 0% 근처여야 합니다. 그리고 측정 중에 브라우저나 IDE를 켜면 그게 "다섯 번째 부하"가 되므로 끕니다.

반복과 기록. 상황마다 최소 3회(r1, r2, r3). 매번 `configs/`에 측정 기록 양식을 채웁니다. 특히 stress-ng의 정확한 옵션, AI 모델 이름, 부하를 켠 시각을 적어야 나중에 같은 실험을 재현할 수 있습니다.

결과를 어떻게 모으고 무엇이 "성공"인가

각 파일에서 A2 계산을 그대로 하고, 표 한 장에 상황별 한 줄씩 넣습니다. Jiao 논문의 Table IV 형식입니다.

상황	격리	avg	p99	p99.9	max	100µs 초과	미스
idle	ccx	3	8	12	25	0	0
stress	ccx	3	9	15	31	0	0
ai-gpu	ccx	4	11	18	38	0	0
full	ccx	4	12	20	42	0	0

위 숫자는 예시입니다. **성공의 모양**은 네 줄의 max가 전부 100 아래이고 서로 비슷한 것입니다. 열이 아래로 갈수록 조금 커지는 건 자연스럽지만, 갑자기 몇 배로 뛰면 그 상황의 간섭 경로가 뚫린 겁니다. 예를 들어 stress는 괜찮은데 ai-gpu에서만 튀면 "GPU 드라이버 인터럽트가 CPU 4번에 떨어지고 있나"를 `/proc/interrupts`에서 확인하는 식으로 범인을 좁힙니다.

실패의 모양은 이미 봤습니다. 팀 베이스라인(일반 커널, 격리 없음)이 idle 146 → stress 1325µs였습니다. 부하를 켜자 9배로 뛴 것, 이게 "분리 안 됨"의 전형입니다. 이 표의 맨 위에 베이스라인 줄을 넣으면 before/after가 그대로 보입니다.

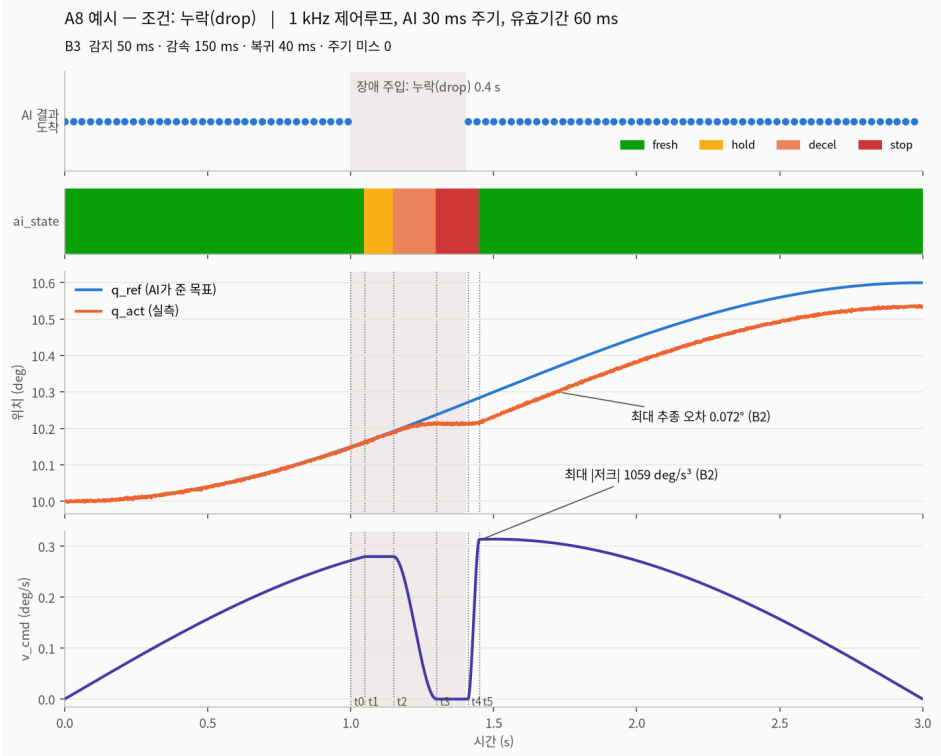
그림은 두 장입니다. 첫째, 네 상황의 히스토그램을 색만 다르게 한 그림에 겹칩니다. 네 산이 겹쳐 보이면 성공입니다. 둘째, 시간축 그림. 가로축이 시간(0~10분), 세로축이 "그 1초 동안 100µs를 넘은 횟수"이고, 부하를 켠 시점에 세로선을 긋습니다. 선 오른쪽에서도 바닥에 붙어 있으면 "부하를 켜도 안 튄다"가 한눈에 보입니다. Jiao 논문의 Fig. 7이 정확히 이 그림입니다.

한 단계 더: 격리 조건도 변수로. 지금 컴퓨터는 CCX 격리가 적용된 상태인데, 커널 파라미터를 바꿔 재부팅하면 "격리 없음", "isolcpus만", "CCX 통째"의 세 상태를 만들 수 있습니다. 네 상황 × 세 격리 = 12칸짜리 표가 되고, "격리를 한 단계씩 올릴수록 부하 상황의 max가 어떻게 내려가는지"가 보입니다. 이건 Jiao 논문도 안 한 측정이라 우리 리포트의 차별점이 됩니다. 다만 재부팅이 필요하니 팀 일정과 맞춰야 합니다.

지금 당장 할 수 있는 건 상황 1과 2입니다. AI 모델 없이 cyclictest와 stress-ng만으로 되고, 둘 다 이미 설치되어 있습니다. 이 두 줄만 나와도 "RT 커널 + 격리 적용 후 부하 중 max"라는 과제 핵심 숫자 하나가 확보됩니다.

---

a8 해석법



A8 그림의 예시입니다. 실제 측정값이 아니라 **"이렇게 생겨야 한다"를 보여 주기 위해 만든 합성 데이터**이고, 그리는 코드는 validation/scripts/plot_a8_example.py에 두었습니다. 나중에 진짜 로그가 오면 데이터 만드는 함수만 CSV 읽기로 바꾸면 같은 그림이 나옵니다.

그림 읽는 법: 위에서 아래로 네 칸

이 그림 한 장이 일기장 세 권(제어루프, AI, 장애)을 한 시간축 위에 겹쳐 놓은 것입니다. 가로축은 전부 같은 시간(0~3초)이고, 회색 띠가 "우리가 일부러 고장 낸 구간"입니다.

1칸, AI 결과 도착. 파란 점 하나가 "머리가 말 한 번 한 것"입니다. 30ms마다 찍히다가 1.0초에 끊기고 1.4초에 다시 시작합니다. 이 칸은 장애 일기(`fault.csv`)와 AI 일기(`ai.csv`)에서 나옵니다. 점이 끊긴 구간이 회색 띠와 일치하는지가 "장애 주입이 제대로 됐나"의 확인입니다.

2칸, 손의 상태 띠. 제어루프 일기의 `ai_state` 열을 색으로 칠한 겁니다. 초록(fresh)이다가 노랑(hold) → 주황(decel) → 빨강(stop) → 다시 초록. 색이 바뀌는 시점이 점선 t1~t5입니다. 이 띠가 **"예외처리가 설계대로 작동했다"의 증거**입니다. 설계가 "유효기간 60ms 지나면 hold, 100ms 더 지나면 감속, 멈춘 뒤 새 말 오면 복귀"라면, 띠의 색 순서와 길이가 그대로 나와야 합니다.

3칸, 위치. 파란 선이 AI가 준 목표 `q_ref`, 주황 선이 실제 위치 `q_act`. 정상 구간에선 둘이 붙어 가다가, 장애 구간에서 주황 선이 멈춥니다(로봇이 안전하게 정지). 그래서 파란 선과 벌어지는데, 그 벌어진 최대 거리가 **B2의 추종 오차**(여기선 0.072°)입니다. 이 그림에서 눈여겨볼 점은 **주황 선이 "점프"하지 않는다**는 것입니다. 나쁜 시스템은 복구 순간 주황 선이 파란 선으로 순간이동합니다.

4칸, 명령 속도. 보라 선이 손이 로봇에 보낸 속도 `v_cmd`입니다. 장애 구간을 보면 t1까지 직전 속도를 그대로 유지(hold), t2~t3에서 S자로 부드럽게 0까지 내려감(decel), t3~t4에서 0(stop), t4~t5에서 다시 올라감(복귀). **이 곡선의 모양이 "급격하지 않다"의 직접 증거**입니다. 수직으로 떨어지면 급정지, 수직으로 올라가면 급발진입니다.

이 그림에서 숫자를 뽑는 법 (B2, B3)

그림 맨 위 한 줄이 B3입니다. 점선 사이 간격을 재면 됩니다.

구간	뜻	이 예시
t0 → t1	장애 시작부터 손이 알아채기까지	50 ms (유효기간 60ms 설계와 일치)
t2 → t3	감속 시작부터 완전 정지까지	150 ms
t4 → t5	새 말 도착부터 정상 복귀까지	40 ms

구간	뜻	이 예시
그 사이	1ms 박자를 놓친 횟수	0

B2는 3칸과 4칸의 화살표입니다. 추종 오차는 두 선의 최대 간격, 저크는 보라 선을 세 번 미분한 최댓값입니다.

이 예시가 보여 주는 교훈 하나

4칸의 최대 저크(1059 deg/s³) 화살표가 **감속 구간이 아니라 복귀 구간(t4~t5)** 을 가리킵니다. 감속은 150ms에 걸쳐 부드럽게 했는데 복귀는 40ms 만에 올려서, 복귀 쪽이 더 "덜컹"한 겁니다. 실제 측정에서도 이런 식으로 **그림이 설계의 약점을 알려 줍니다.** 이 경우 리포트에 "복귀 블렌딩 시간을 늘려야 한다"고 쓰면 됩니다. 또 하나, 3칸에서 복구 후에도 주황 선이 파란 선을 따라잡지 못하고 평행하게 가는데, 이건 이 합성 예시가 "멈춘 동안 뒤처진 만큼"을 보정하지 않아서입니다. 팀의 진행도 추적 정책은 실제 위치를 경로에 다시 투영해서 이어 가므로, 진짜 측정에선 이 간격이 줄어들어야 하고 그게 바로 반영 정책 비교 실험에서 보여 줄 차이입니다.

다섯 조건에서 각각 어떻게 다르게 보이나

같은 틀로 조건마다 한 장씩, 총 다섯 장을 만듭니다.

- **정상**: 회색 띠 없음, 2칸 전부 초록, 3칸 두 선이 끝까지 붙어 감.
- **지연**: 1칸 점 간격이 넓어짐(끊기진 않음). 지연이 유효기간 안이면 2칸은 초록 유지, 넘으면 짧은 노랑이 간헐적으로 나타남.
- **누락**: 위 그림.
- **이상값**: 1칸 점은 정상인데 2칸에 회색(reject) 조각이 섞임. 3칸은 흔들리지 않아야 함. 엉뚱한 값이 들어와도 손이 거부해서 위치가 안 튀는 게 증거.
- **통신단절**: 누락과 비슷하지만 감지 경로가 다름(유효기간이 아니라 통신 이상 감지). t0→t1이 더 짧아야 정상.

다섯 장을 나란히 놓으면 심사위원이 "어떤 고장이 와도 4칸 곡선이 부드럽다"를 한눈에 확인할 수 있습니다.
